Server-side jQuery and more cool tricks with Aptana Jaxer

In this example, (the first in a series) we're going to build a simple voting tool, using a single page of DHTML. The implementation is quite basic but covers a good few examples of how to use Aptana Jaxer in real world situations, such as :

- Using Ajax libraries on the server (jQuery in this example)
- Server-side DOM manipulation (using jQuery)
- Storing and retrieving Session data
- Creating and accessing a database
- E4X as a templating mechanism (E4X provides native XML support in JavaScript)
- Handling form data

Let's Vote...

This example was written as a single page webapp. You could remove the Javascript to another file and make it all unobtrusive if that is what gets you excited, but I'm just using an inline script tag, as the code is only about 30 or so lines long. Also in Jaxer we could have easily implemented this using Jaxer's seamless ajax callback mechanism but for the purpose of this example we'll stick with a traditional form post.

Let's get started. Most folks reading this should be familiar with the standard blog/portal poll, where you are presented with a set of choices.

and once you have voted you get to see the current results of the voting.

In our example we allow multiple votes per user but you can easily change that by just commenting out a single line of code.

One of the interesting features of this application is that, by using server-side DOM manipulation in Jaxer, you can remove any unwanted content before it is sent to the client browser. We use this to hide the vote results until the user has voted

This is a useful technique for permission based forms where, for example, you may want remove the credit card details unless the user has established the correct permissions and been validated by the server.

A very basic web page

So let's jump in and have a look at the code used to make this work. **The code in this article has been updated to use the API of Jaxer 0.9.8 or later, for The full source listing below contains comments to indicate where the code was changed to work with the later API.**

```
01. <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1"/>
02. <title>Jaxer Poll</title>
03. <script runat="server" src="http://jaxer.org/tutorials/jquery-compressed.js" autoload="true"></script>
04. <style>
05.         #pollContainer {
06.             width: 600px;
07.             background: #eee;
08.             border: 3px solid #555;
09.             padding: 5px;
10.             font-size: 1em;
11.             font-family: Tahoma, Verdana, Helvetica, Arial;
12.             font-weight: 900;
13.         }
14.
15.         #display {
16.             background: #ccc;
17.             font-size: 0.75em;
18.             font-weight: 200;
19.             color: #2E3540;
20.             margin: 5px 0 5px 0;
21.         }
22.
23.         TD {
24.             padding-left: 10px;
25.         }
26.
27.         TD IMG {
28.             margin-left: -6px;
29.         }
30. </style>
```

Above is the contents of the HEAD element. Just the usual suspects, setting the title and some simple CSS stuff. The only interesting part is at line #3, where we load the jQuery library on the **server**, because we intend to do some **serverside DOM manipulation** before the page is sent to the client.

The runat='server' attribute tells Jaxer to load this javascript library **on the server**.

The autoload attribute is a recent addition to Jaxer, and it instructs Jaxer to load that page and keep it cached as preparsed bytecode (for faster library loadtime) for any future calls to this page, including callbacks.

In the body of the document we have a simple form which we will dynamically populate on the server. The form will post the vote to itself on the server.

We are marking up the DOM content with the classnames 'voter' and 'nonvoter' to identify content that is specific to a user's status, and make it easily accessible using jQuery on the server.

```
<form id="pollContainer" method="POST" accept-charset="utf-8">
      <div id="question">What do you think is the coolest feature of Jaxer?</div>
      <div id="display"></div>
      <div class="nonvoter"><input type="submit" value="vote"></div><div id="voteTotals" class="voter"></div>
  </form>
```

Server Side Javascript

Now we get to the Javascript. The app has a single script tag in the body which is designated to run on the server.

The first dozen or so lines are **simply creating a DB and preparing a schema** for use.

```
var db = new Jaxer.DB.SQLite.createDB({PATH:Jaxer.Dir.resolve('poll.sqlite')});
  db.execute("CREATE TABLE IF NOT EXISTS votes (id, ip, vote)");

  var questions = [ "Server-Side DOM","Server-Side Javascript","Seamless Ajax","I already know it!"];

  var totalVotes = db.execute('SELECT count(*) FROM votes').singleResult - questions.length;
  if (totalVotes < 0)
  {
      questions.map(function(item, index) // seed the table with 1 row per question to avoid nulls
    {
        db.execute("INSERT INTO votes (id ,ip,vote) VALUES (?,?,?)", ""+index, '127.0.0.1', ""+index);
      });
    totalVotes = 0;
  }
```

So we've connected to the database (which was automatically created if needed, how convenient!) and set up the schema and initial content we expect. We also setup an Array (***questions***) containing the questions for the voting poll

Next we need to check the **session value** we are storing (***status***) to determine whether this user has already voted, and then check to see if the form data for the vote is being posted. Then, if we are actually voting, we write the vote to the database and set the user status to 'voter'.

When we write the vote to the database, we grab the sessionID and the remote IP address and write those out with the vote data, this will let us enforce single voting later if we need it.

Finally query the database to get the current vote counts, remembering to subtract 1 from the total to account for the dummy rows we inserted to seed database and prevent any nulls from appearing in the results totals.

```
var status = Jaxer.session.status || 'nonvoter';

  // look for request data and save vote if present
  if (Jaxer.request.data.hasOwnProperty('radioBtns') && status == 'nonvoter')
  {
    db.execute("INSERT INTO votes (id,ip,vote) VALUES (?,?,?) "
    ,   Jaxer.SessionManager.keyFromRequest()   // session ID
    ,   Jaxer.request.remoteAddr                // remote IP address
    ,   Jaxer.request.data.radioBtns            // posted data
    );
      status = 'voter';
    totalVotes ++ ;
  }
  rs = db.execute('SELECT count(*)-1 AS tot FROM votes GROUP BY vote');
```

Now we build the DOM, to do this we issue a query to the DB to get the current vote tally.

Using E4X - ECMAScript For XML as a simple templating tool we create the DOM with the nodes populated according to our dataset.

One of the nice features of server-side javascript with Jaxer **is that you have access to all neat things built into Mozilla without the worry of client side browser support**, which enables reliable use of the advanced features of the Javascript language.

If you look closely at the code below you'll notice we use a simple syntax for variable substitution using curly braces containing javascript code inside the E4X assigments. This allows us to use this for templating as long as we are creating valid XML nodes.

```
var output = <table></table>;
  for (question in questions)
  {
    var percent = parseInt((rs.rows[question].tot / totalVotes) * 100);
      output.appendChild(<tr>
      <td class="nonvoter"><input type="radio" name="radioBtns" value="{question}"></td>
      <td>{questions[question]}</td>
      <td class="voter">
           <img src="http://jaxer.org/tutorials/poll_l.png" height="16">
           <img src="http://jaxer.org/tutorials/poll.png" height="16" width="{percent" * 2}>
           <img src="http://jaxer.org/tutorials/poll_r.png" height="16"></td>
      <td class="voter">{rs.rows[question].tot}</td>
      <td class="voter">({percent}%)</td>
      </tr>);
  }
  $('#display').html(output.toString());
  $('#voteTotals').html(totalVotes + " votes");
```

So the document now has a DOM that has been populated with the content for **BOTH** voters and non-voters. We use a little **jQuery** magic to remove the elements we don't want presented to the user.

```
// remove DOM elements based on whether user has already voted
  $((status == 'voter') ? '.nonvoter' : '.voter').remove();
```

In this way the user will EITHER see the form with the question and the submit button

OR the current voting results data.

Now we set the **session variable** status to be the current status of the user as they have either voted or not.

```
Jaxer.session.status = status;
```

Finally as the page that is served has no further dependency on Jaxer once it leaves the server, we tell Jaxer to not bother injecting the client framework. Normally the client framework would be automatically inserted as a script tag in the outgoing HTML, but our simple example doesn't need to communicate back to the Jaxer server, as it contains no server callbacks.

```
// no need for the client Framework to be injected into the page
Jaxer.response.setClientFramework();
```

So there you have it, a simple poll on a single page, using many of Jaxer's cool features.

Supporting Files

The full code and supporting files for this article are available here

Full Source Listing

```
<html>
  <head>
    <title>Jaxer Poll</title>
    <script runat="server" src="http://jaxer.org/tutorials/jquery-compressed.js" autoload="true">
    </script>
    <style>
      #pollContainer {
        width: 600px;
        background: #eee;
        border: 3px solid #555;
        padding: 5px;
        font-size: 1em;
        font-family: Tahoma, Verdana, Helvetica, Arial;
        font-weight: 900;
      }

      #display {
        background: #ccc;
        font-size: 0.75em;
        font-weight: 200;
        color: #2E3540;
        margin: 5px 0 5px 0;
      }

      TD {
        padding-left: 10px;
      }

      TD IMG {
        margin-left: -6px;
      }
    </style>
  </head>
  <body>
    <form id="pollContainer" method="POST" accept-charset="utf-8">
      <div id="question">
        What do you think is the coolest feature of Jaxer?
      </div>
      <div id="display">
      </div>
      <div class="nonvoter">
        <input type="submit" value="vote"/>
      </div>
      <div id="voteTotals" class="voter">
      </div>
    </form>
    <script runat="server">
      // for the beta version of Jaxer, change it to the following
      //var db = Jaxer.DB.SQLite.createDB({PATH:Jaxer.Dir.resolvePath('poll.sqlite')});
      var db = Jaxer.DB.SQLite.createDB({PATH:Jaxer.Dir.resolve('poll.sqlite')});
      db.execute("CREATE TABLE IF NOT EXISTS votes (id, ip, vote)");

      var questions =
      [
        "Server-Side DOM"
      , "Server-Side Javascript"
      , "Seamless Ajax"
      , "I already know it!"
      ];

      var totalVotes = db.execute('SELECT count(*) FROM votes').singleResult -
        questions.length;
      if (totalVotes < 0)
      {
        // seed the table with 1 row per question to avoid nulls
        questions.map(function(item, index)
        {
          db.execute("INSERT INTO votes (id ,ip,vote) VALUES (?,?,?)",
            ""+index, '127.0.0.1', ""+index);
        });
        totalVotes = 0;
      }
      // remove the following line to limit to 1 vote per user session
      // for the beta version of Jaxer, change it to the following
      //Jaxer.session.remove('status');
      delete Jaxer.session.status;

      // for the beta version of Jaxer, change the following line to
      //var status = Jaxer.session.get('status') || 'nonvoter';
      var status = Jaxer.session.status || 'nonvoter';

      // look for request data and save vote if present
      if ('radioBtns' in Jaxer.request.data &&
          (status == 'nonvoter'))
      {
        db.execute("INSERT INTO votes (id,ip,vote) VALUES (?,?,?) "
        ,   Jaxer.SessionManager.keyFromRequest()   // session ID
        ,   Jaxer.request.remoteAddr        // remote IP address
        ,   Jaxer.request.data.radioBtns      // posted data
        );
        status = 'voter';
```

```
        totalVotes ++ ;
      }
      rs = db.execute('SELECT count(*)-1 AS tot FROM votes GROUP BY vote');

      var output = <table></table>;
      for (var question in questions)
      {
        var percent = parseInt((rs.rows[question].tot / totalVotes) * 100);
        output.appendChild(
          <tr>
            <td class='nonvoter'>
              <input type='radio' name='radioBtns' value={question}/>
            </td>
            <td>{questions[question]}</td>
            <td class='voter'>
              <img src="poll_l.png" height='16'/>
              <img src="poll.png"   height='16' width={2+(percent * 2)}/>
              <img src="poll_r.png" height='16'/></td>
            <td class='voter'>{rs.rows[question].tot}</td>
            <td class='voter'>({percent}%)</td>
          </tr>);
      }
      $('#display').html(output.toString());
      $('#voteTotals').html(totalVotes + " votes");

      // remove DOM elements based on whether user has already voted
      $((status == 'voter') ? '.nonvoter' : '.voter').remove();

      // for the beta version of Jaxer, change the following line to
      //Jaxer.session.set('status', status);
      Jaxer.session.status = status;

      // no need for the client Framework to be injected into the page.
      Jaxer.response.setClientFramework();
    </script>
  </body>
</html>
```